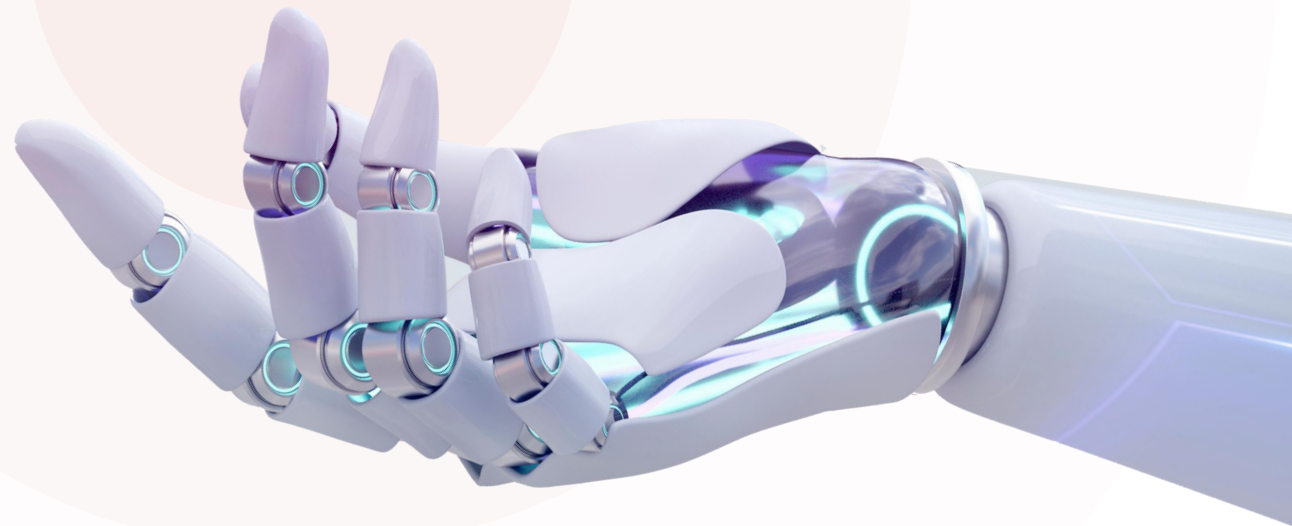


Python核心语法

AI Copilot



多一句没有，少一句不行，用更短的时间，教会更实用的技术



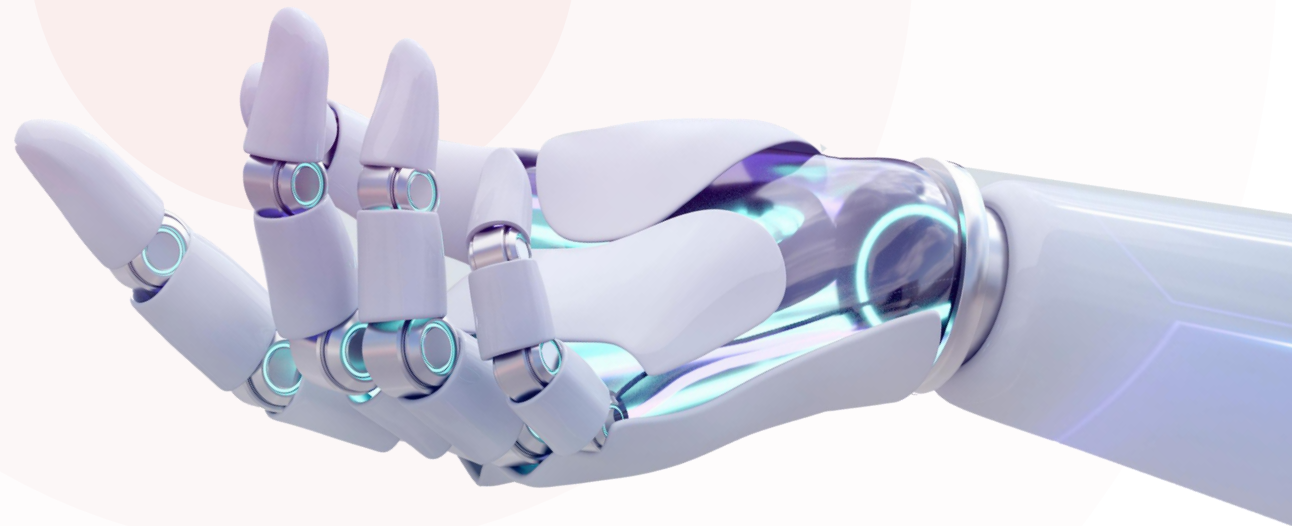
Python核心语法



Python核心语法

—— 函数 ——

AI Copilot



多一句没有，少一句不行，用更短的时间，教会更实用的技术

什么是函数？

- 函数是组织好的、可重复使用的、用来实现特定功能的代码片段。



```
goods_name = input("请输入商品名称：")
goods_price = float(input("请输入商品价格："))
goods_num = int(input("请输入商品数量："))

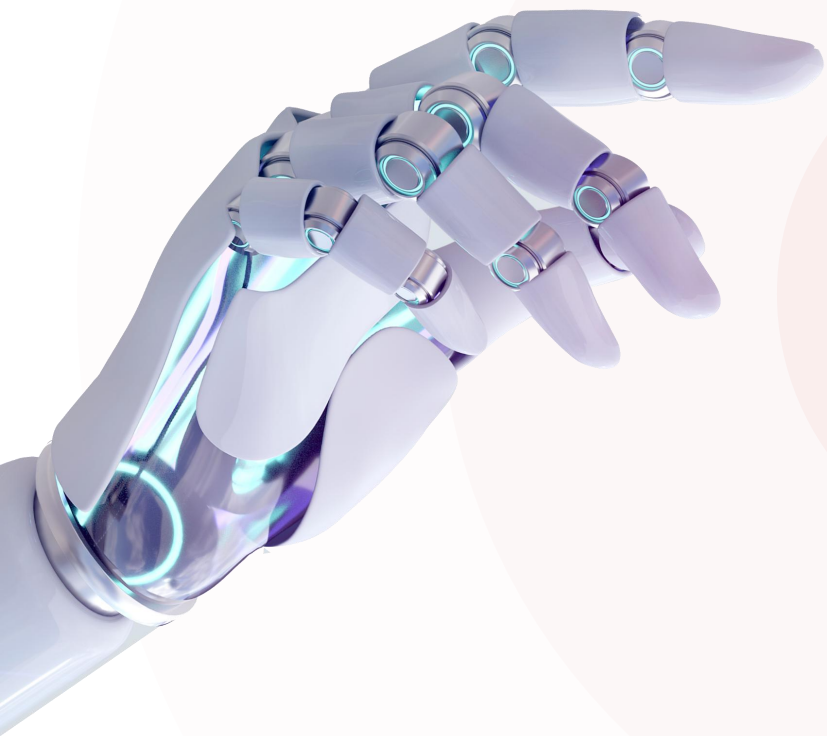
# 如果商品存在，则不执行添加，提示信息
if goods_name in shopping_cart:
    print("该商品已存在，请重新选择 ~")
else:
    shopping_cart[goods_name] = {"price": goods_price, "num": goods_num}
    print("商品添加完毕 ~")
```

- 要输入，直接调用input()
- 要输出，直接调用print()

max() min() len() sum()

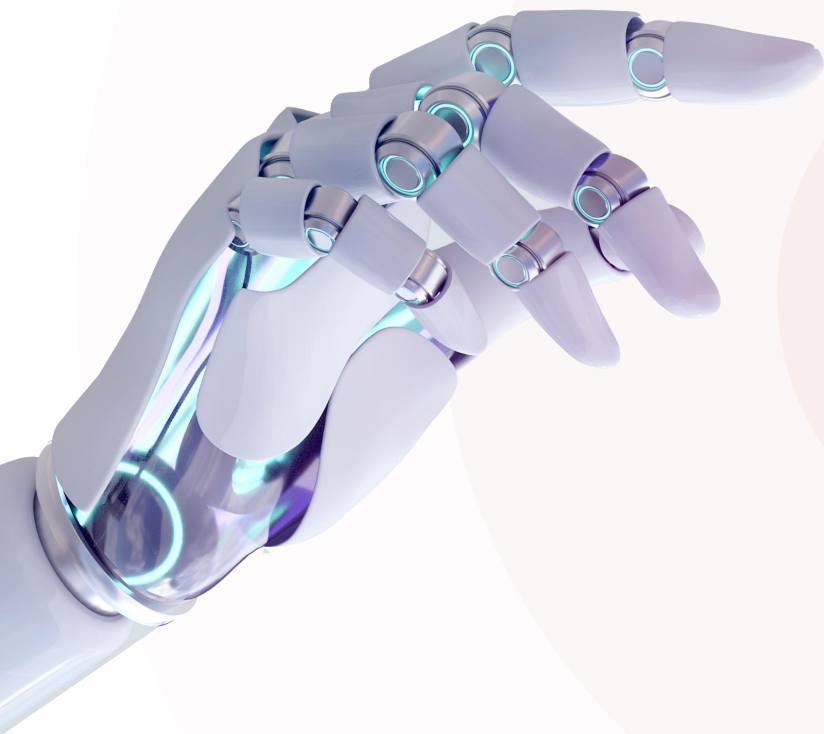
因为 input()、print() 是Python的内置函数：

- 是提前定义好的
- 可以重复使用
- 实现特定功能的



CONTENT 目录

- ◆ 函数基础
- ◆ 函数进阶



1

函数基础

- ◆ 函数定义
- ◆ 函数的参数与返回值
- ◆ 函数说明文档
- ◆ 案例

函数

- 函数是组织好的、可重复使用的、用来实现特定功能的代码片段。
- 函数定义与调用：

```
# 定义函数
def 函数名(参数列表):
    函数体
    .....
    return 返回值

# 调用函数
函数名(参数)
```

```
# 定义函数
def out_line():
    print('-----')

# 调用函数
out_line()
```

注意：函数定义时的参数列表与返回值语句是可有可无的(由需求确定)，函数必须先定义后调用执行（函数定义时，内部代码并未执行，在调用时才执行）。

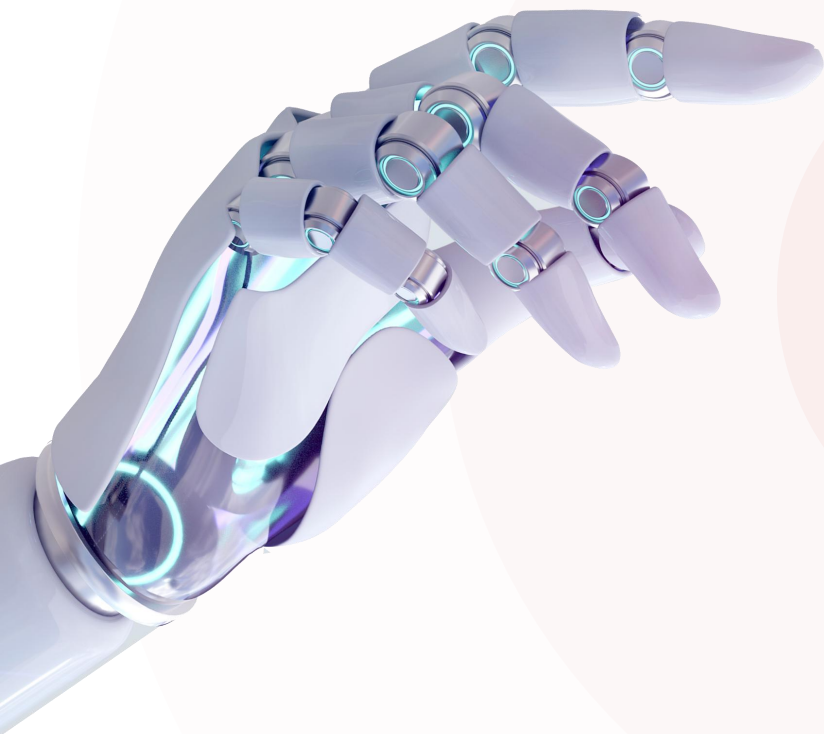
BRIEF 小结

1. 函数定义及调用的语法？

```
# 函数的定义  
  
def out_line():  
    print('-----')  
  
# 函数的调用  
  
out_line()
```

2. 函数使用的注意事项

- 函数必须先定义，在调用
- 函数定义时，并不会执行，只有在调用函数时，函数体的逻辑才会运行
- 函数中通过缩进来描述归属关系



1

函数基础

- ◆ 函数定义
- ◆ 函数的参数与返回值
- ◆ 函数说明文档
- ◆ 案例

函数的参数与返回值

- 在定义函数时，根据业务需要，可以指定参数与返回值，具体格式如下：

定义函数

def 函数名(参数列表):

函数体

.....

return 返回值

调用函数

函数名(参数)

计算圆的面积

def circle_area(r):

area = 3.14 * r * r

return area

调用函数

c_area = circle_area(10)

print(c_area)

计算长方形的面积

def rectangle_area(l, w):

area = l * w

return area

调用函数

r_area = rectangle_area(20, 10)

print(r_area)

• 实参（实际参数）：函数在实际调用时传入的参数

• 形参（形式参数）：函数定义时括号里的参数，只能在函数内使用(局部变量)

注意：函数定义时如果有多个参数，多个参数之间使用逗号(,)分隔。

注意：return语句只有返回功能，而没有输出打印的功能，如果要输出，需要结合print()函数来实现。

小结

1. 有参数有返回值的函数定义及调用？

```
def rectangle_area(l, w):  
    return l * w  
  
r_area = rectangle_area(l: 20, w: 10)  
print(r_area)
```

形参

实参

2. 函数可以有多个返回值吗？

```
def circle_area_len(r):  
    return 3.14 * r * r, 2 * 3.14 * r  
  
a1 = circle_area_len(10)  
print(a1)
```

封装到元组中

```
def circle_area_len(r):  
    return 3.14 * r * r, 2 * 3.14 * r  
  
area, len = circle_area_len(10)  
print(area, len)
```

元组解包

思考

```
def rectangle_area(l,w):  
    area = l * w  
    return area
```

```
print(rectangle_area(l: 20, w: 10))
```

不友好

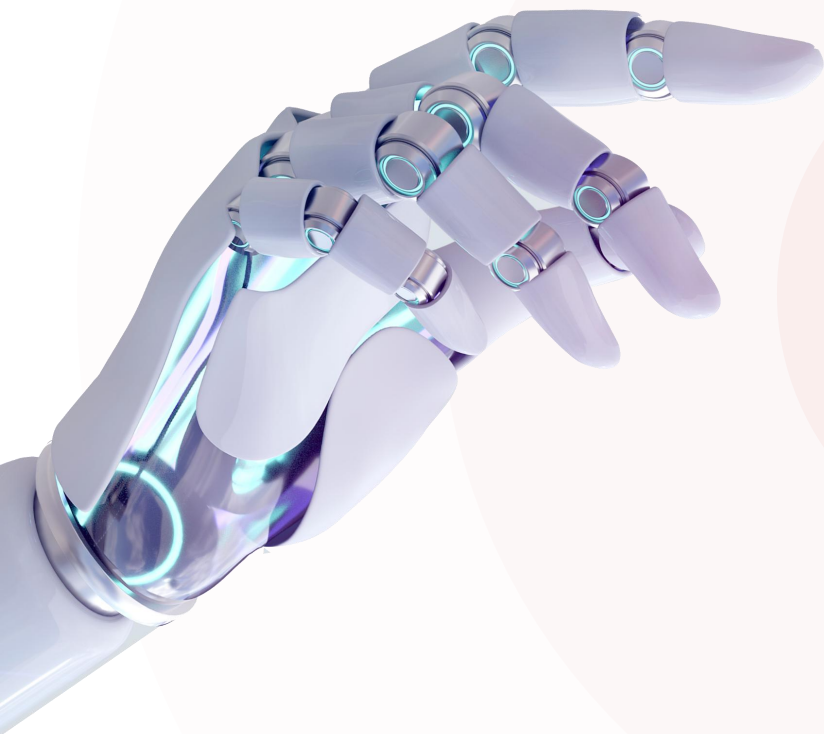
难理解

难维护

```
def circle_area_len(r):  
    return round(3.14 * r * r, 1), round(2 * 3.14 * r, 1)
```

```
al = circle_area_len(10)  
print(al)
```

函数说明文档



1

函数基础

- ◆ 函数定义
- ◆ 函数的参数与返回值
- ◆ 函数说明文档
- ◆ 案例

函数的说明文档

- 函数的说明文档（Docstring）是写在函数开头，用三个引号包裹的字符串，用于解释函数的功能、参数、返回值等信息，方便调用者清楚函数的具体作用及细节。

```
# 定义一个函数, 根据半径, 计算圆的周长、面积
def circle_area_len(r):
    """
    该函数用于根据圆的半径, 计算圆的面积和圆的周长

    :param r: 圆的半径
    :return: 圆的面积, 圆的周长
    """
    return 3.14 * r * r, 2 * 3.14 * r

al = circle_area_len(10)
print(al)
```

→ 描述函数的返回值

→ 描述函数的参数

查看函数说明文档:

- 使用help函数，比如：help(circle_area_len)
- 鼠标悬浮在函数上，自动展示（推荐）

```
al = circle_area_len(10)
```

```
print(al)
```

PEP 8: E305 expected 2 blank lines after class or function definition, found 1

Reformat the file Alt+Shift+Enter More actions... Alt+Enter

D:/py_code_workspace/py_project1/04. 函数.py

def circle_area_len(r: Any) -> tuple[float | Any, float | Any]

该函数用于根据圆的半径, 计算圆的面积和圆的周长

Params: r - 圆的半径

Returns: 圆的面积, 圆的周长

记住：好的文档，能让你的代码更容易理解、使用和维护！

函数的嵌套调用

- 嵌套调用指的是在一个函数中，又调用了另外一个函数。
- 函数调用遵循栈结构，最后被调用的函数最先返回LIFO(Last In First Out, 后进先出)

```
def function_a():  
    print("a ... before")  
    function_b()  
    print("a ... after")
```

```
def function_b():  
    print("b ... before")  
    function_c()  
    print("b ... after")
```

```
def function_c():  
    print("c ...")
```

```
function_a()
```

入栈

出栈

function_a

function_a() 开始

|
|—— 打印 "a ... before"

|
|—— 调用 function_b()

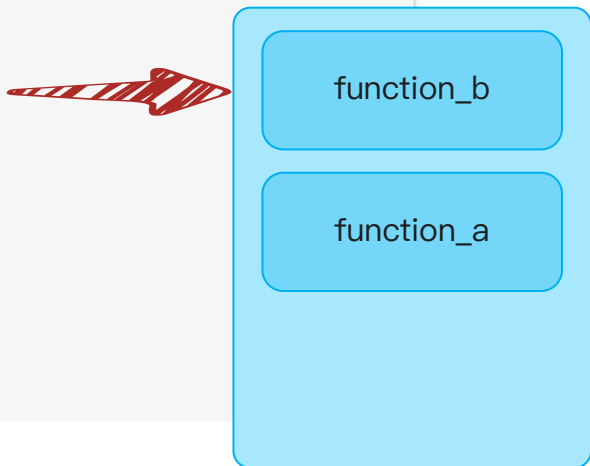
- 嵌套调用指的是在一个函数中，又调用了另外一个函数。
- 函数调用遵循栈结构，最后被调用的函数最先返回LIFO(Last In First Out, 后进先出)

```
def function_a():
    print("a ... before")
    function_b()
    print("a ... after")
```

```
def function_b():
    print("b ... before")
    function_c()
    print("b ... after")
```

```
def function_c():  
    print("c ...")
```

```
function_a()
```



function_a() 开始

```
|
|— 打印 "a ... before"
|
|— 调用 function_b()
|   |
|   |— 打印 "b ... before"
|   |
|   |— 调用 function_c()
|   |
|   |
```


函数的嵌套调用

- 嵌套调用指的是在一个函数中，又调用了另外一个函数。
- 函数调用遵循栈结构，最后被调用的函数最先返回LIFO(Last In First Out, 后进先出)

```
def function_a():  
    print("a ... before")  
    function_b()  
    print("a ... after")
```

```
def function_b():  
    print("b ... before")  
    function_c()  
    print("b ... after")
```

```
def function_c():  
    print("c ...")
```

function_a()



function_c

function_b

function_a

function_a() 开始

| — 打印 "a ... before"

| — 调用 function_b()

| | — 打印 "b ... before"

| | — 调用 function_c()

| | | — 打印 "c ..."

函数的嵌套调用

- 嵌套调用指的是在一个函数中，又调用了另外一个函数。
- 函数调用遵循栈结构，最后被调用的函数最先返回LIFO(Last In First Out, 后进先出)

```
def function_a():  
    print("a ... before")  
    function_b()  
    print("a ... after")
```

```
def function_b():  
    print("b ... before")  
    function_c()  
    print("b ... after")
```

```
def function_c():  
    print("c ...")
```

function_a()



function_b

function_a

function_a() 开始

| — 打印 "a ... before"

| — 调用 function_b()

| | — 打印 "b ... before"

| | — 调用 function_c()

| | | — 打印 "c ..."

| | — 打印 "b ... after"

| | — function_b() 返回

函数的嵌套调用

- 嵌套调用指的是在一个函数中，又调用了另外一个函数。
- 函数调用遵循栈结构，最后被调用的函数最先返回LIFO(Last In First Out, 后进先出)

```
def function_a():  
    print("a ... before")  
    function_b()  
    print("a ... after")
```

```
def function_b():  
    print("b ... before")  
    function_c()  
    print("b ... after")
```

```
def function_c():  
    print("c ...")
```

```
function_a()
```



function_a

function_a() 开始

| — 打印 "a ... before"

| — 调用 function_b()

| | — 打印 "b ... before"

| | — 调用 function_c()

| | | — 打印 "c ..."

| | — 打印 "b ... after"

| | — function_b() 返回

| — 打印 "a ... after"

| — function_a() 返回

函数的嵌套调用

- 嵌套调用指的是在一个函数中，又调用了另外一个函数。
- 函数调用遵循栈结构，最后被调用的函数最先返回LIFO(Last In First Out, 后进先出)

```
def function_a():  
    print("a ... before")  
    function_b()  
    print("a ... after")
```

```
def function_b():  
    print("b ... before")  
    function_c()  
    print("b ... after")
```

```
def function_c():  
    print("c ...")
```

```
function_a()
```

function_a() 开始

| — 打印 "a ... before"

| — 调用 function_b()

| | — 打印 "b ... before"

| | — 调用 function_c()

| | | — 打印 "c ..."

| | — 打印 "b ... after"

| | — function_b() 返回

| — 打印 "a ... after"

| — function_a() 返回

函数的嵌套调用

- 嵌套调用指的是在一个函数中，又调用了另外一个函数。
- 函数调用遵循栈结构，最后被调用的函数最先返回LIFO(Last In First Out, 后进先出)

```
def function_a():  
    print("a ... before")  
    function_b()  
    print("a ... after")
```

```
def function_b():  
    print("b ... before")  
    function_c()  
    print("b ... after")
```

```
def function_c():  
    print("c ...")
```

function_a()



function_a() 开始

—— 打印 "a ... before"

—— 调用 function_b()

—— 打印 "b ... before"

—— 调用 function_c()

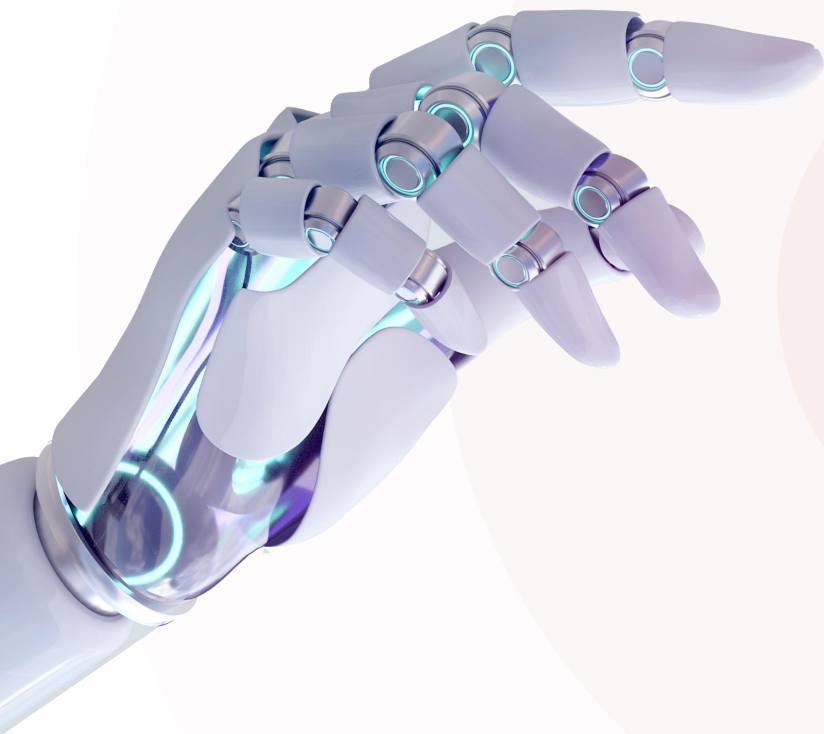
—— 打印 "c ..."

—— 打印 "b ... after"

—— function_b() 返回

—— 打印 "a ... after"

—— function_a() 返回



1

函数基础

- ◆ 函数定义
- ◆ 函数的参数与返回值
- ◆ 函数说明文档
- ◆ 案例



案例

完成如下需求



1. 定义一个函数：根据传入的底和高计算三角形面积的函数（三角形面积 = 底 * 高 / 2）。
2. 定义一个函数：计算传入的字符串中元音字母的个数（元音字母为 aeiouAEIOU）。
3. 定义一个函数：计算传入的班级学员高考成绩列表中成绩的最高分、最低分、平均分(保留1位小数)，并返回。

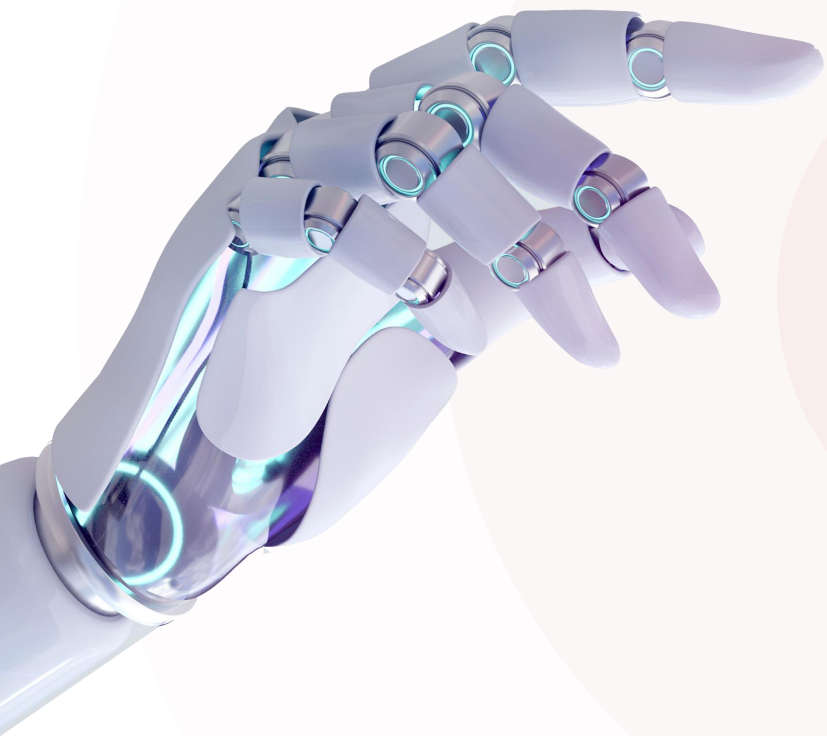


练习

完成如下需求



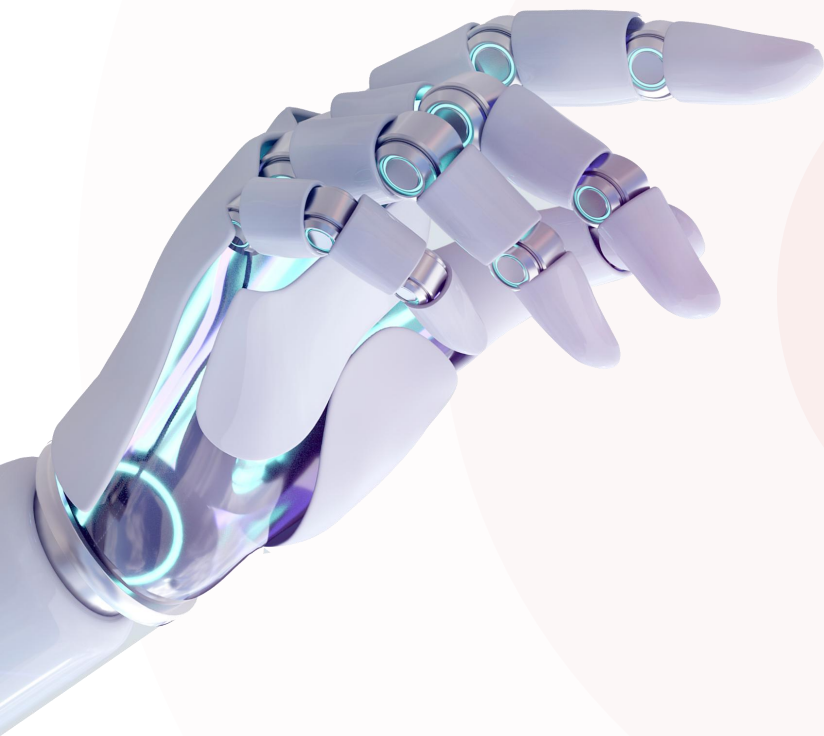
1. 定义一个函数，根据传入的分数，计算对应的分数等级并返回。
 - 分数 ≥ 90 : A
 - 分数 ≥ 75 : B
 - 分数 ≥ 60 : C
 - 分数 < 60 : D
2. 定义一个函数，用于判断一个字符串是否是回文串，返回bool值。
 - 把字符串反转，如果和原字符串相同，就是回文串。（如："level", "radar", "黄山落叶松叶落山黄"）
3. 定义一个函数：完成时间转换功能，将传入的秒转换为小时、分钟、秒。
4. 定义一个函数：根据传入的三角形三个边的边长，判定三角形的类型（等边、等腰、普通，或者不能构成三角形）。



CONTENT 目录

◆ 函数基础

◆ 函数进阶



2

函数进阶

- ◆ 函数变量的作用域
- ◆ 函数参数详解
- ◆ 匿名函数
- ◆ 案例

函数-变量作用域

- 变量的作用域指的是变量的作用范围(标识这个变量在哪里可以使用，在哪儿不可以使用)。

```
# 定义函数
```

```
num = 100
```

全局变量

```
def circle_area(r):
```

```
    pi = 3.14
```

```
    area = pi * r * r
```

局部变量

```
    return area
```

```
count = 0
```

```
# 调用函数
```

全局变量

```
c_area = circle_area(10)
```

```
print(c_area)
```

全局变量：在函数之外定义的变量，称之为全局变量，在整个文件中（包括函数内）都可以使用（通常定义在文件的顶部）。

局部变量：在函数内部定义的变量，称之为局部变量，只能在该函数内部使用，外部无法访问(函数执行完毕后，会自动销毁其内部局部变量)。

global关键字

- global关键字用于明确的告诉Python解释器，在函数中要使用全局变量，使得可以在函数内部修改全局变量的值。

```
num1 = 1 # 全局变量

def fun1():

    num1 = 100 # 局部变量

    print(num1)

fun1() # 100

print(num1) # 1
```

```
num1 = 1 # 全局变量

def fun1():

    global num1    告诉python解释器，函数中使用全局变量num1

    num1 = 100 # 修改全局变量num1

    print(num1)

fun1() # 100

print(num1) # 1
```

注意：在基于global声明全局变量时，要先声明，再使用。

BRIEF 小结

1. 什么是局部变量，全局变量？

- 在函数内部定义的变量就是局部变量，函数外声明的

2. global关键字的作用？

- 在函数内部使用，声明接下来要使用的是全局变量，

3. 注意事项

- 尽量避免在函数中使用全局变量，因为会使代码难以
- 考虑使用函数参数和返回值来传递数据，而不是依赖
- global主要用在程序的状态、配置和计数器等场景中

调试开关

debug_mode = False

def enable_debug_mode():

 global debug_mode

 debug_mode = True

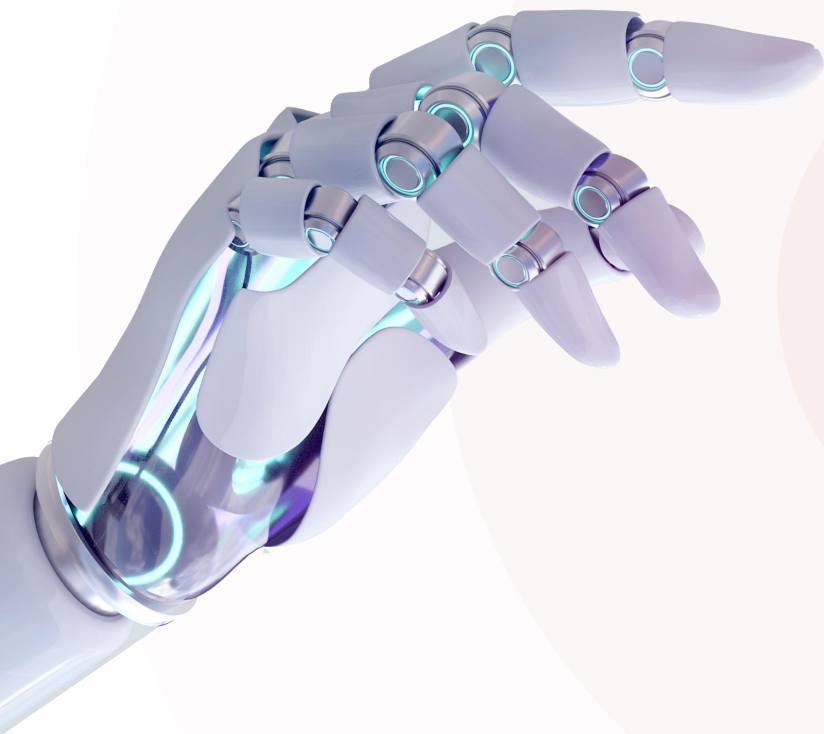
 print("调试模式已开启")

def disable_debug_mode():

 global debug_mode

 debug_mode = False

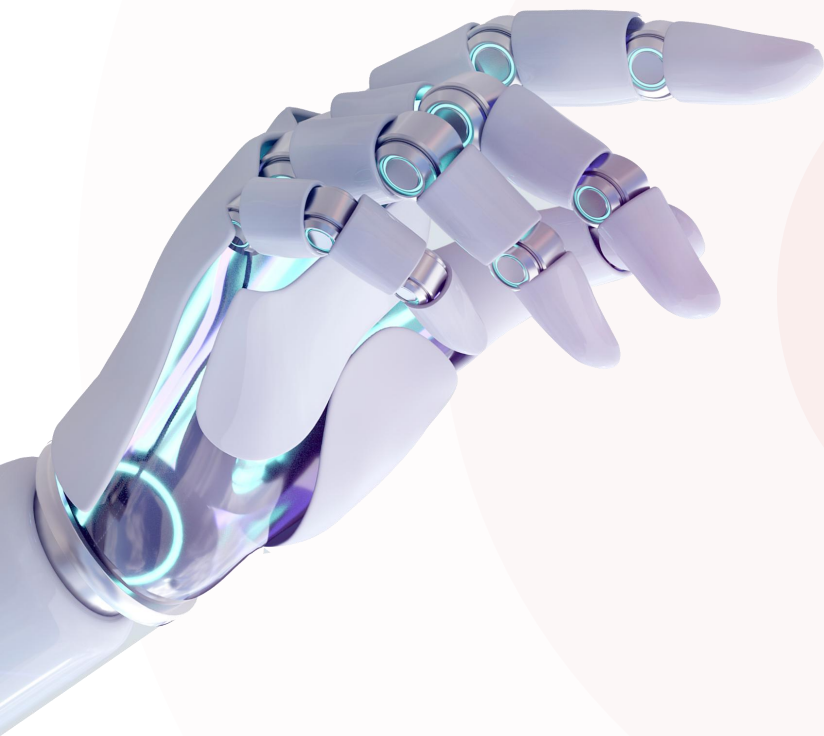
 print("调试模式已关闭")



2

函数进阶

- ◆ 函数变量的作用域
- ◆ 函数参数详解
- ◆ 匿名函数
- ◆ 案例



2

函数进阶

- ◆ 函数变量的作用域
- ◆ 函数参数详解
 - 传参方式
 - 默认参数
 - 不定长参数
- ◆ 匿名函数
- ◆ 案例

函数-传参方式

- 传参方式指的是，在调用函数时，传递实参的方式。

1. 位置参数：调用函数时根据函数定义时的位置来传递参数。

```
# 定义函数

def reg_stu(name, age, gender, city):
    print(f"注册成功, 姓名:{name}, 年龄:{age}, 性别:{gender}, 城市:{city}")
    return {"name": name, "age": age, "gender": gender, "city": city}

# 调用函数

stu = reg_stu("张三", 18, "男", "北京")
print(stu)
```

```
# 定义函数
def 函数名(参数列表):
    函数体
    .....
    return 返回值

# 调用函数
函数名(参数...)
```



要求：调用函数时参数顺序与定义函数时参数顺序完全一致

函数-传参方式

- 传参方式指的是，在调用函数时，传递实参的方式。

2. **关键字参数**：调用函数时以函数定义时形参名称作为关键字，以 "键=值" 的形式来传递参数(不要求顺序)。

```
# 定义函数
def reg_stu(name, age, gender, city):
    print(f"注册成功, 姓名:{name}, 年龄:{age}, 性别:{gender}, 城市:{city}")
    return {"name": name, "age": age, "gender": gender, "city": city}

# 调用函数

stu = reg_stu(name="张三", age=18, gender="男", city="北京")
print(stu)

stu2 = reg_stu(gender="男", name="王武", city="上海", age=22)
print(stu2)

stu2 = reg_stu("赵四", 28, gender="男", city="上海")
print(stu2)
```

位置参数 **关键字参数**

要求：如果位置参数与关键字参数混用，关键字参数必须在位置参数之后(关键字参数之间,没有顺序要求)

位置参数 vs 关键字参数

位置参数

```
# 定义函数

def reg_stu(name, age, gender, city):
    print(f"注册成功, 姓名:{name}, 年龄:{age}, 性别:{gender}, 城市:{city}")
    return {"name": name, "age": age, "gender": gender, "city": city}

# 调用函数

stu = reg_stu("张三", 18, "男", "北京")
print(stu)
```

优点：简洁

缺点：可读性差、易出错、维护难

场景：参数少（不超过3个），且顺序自然

```
s = calc(87, 68, 92, 85)
```

关键字参数

```
# 定义函数

def reg_stu(name, age, gender, city):
    print(f"注册成功, 姓名:{name}, 年龄:{age}, 性别:{gender}, 城市:{city}")
    return {"name": name, "age": age, "gender": gender, "city": city}

# 调用函数

stu = reg_stu(name="张三", age=18, gender="男", city="北京")

stu2 = reg_stu(gender="男", name="王武", city="上海", age=22)
```

优点：可读性强、易维护和扩展

缺点：代码繁琐

场景：参数较多，或易混淆的场景

```
s = calc(math=87, chinese=68, english=92, computer=85)
```

黄金法则：半年后回头看你今天写的代码，能否一眼看出每个参数的含义，如果不能，就应该使用关键字参数。

小结

1. 位置参数

- 调用函数时，传入的实参的顺序与定义函数时形参的顺序完全一致。

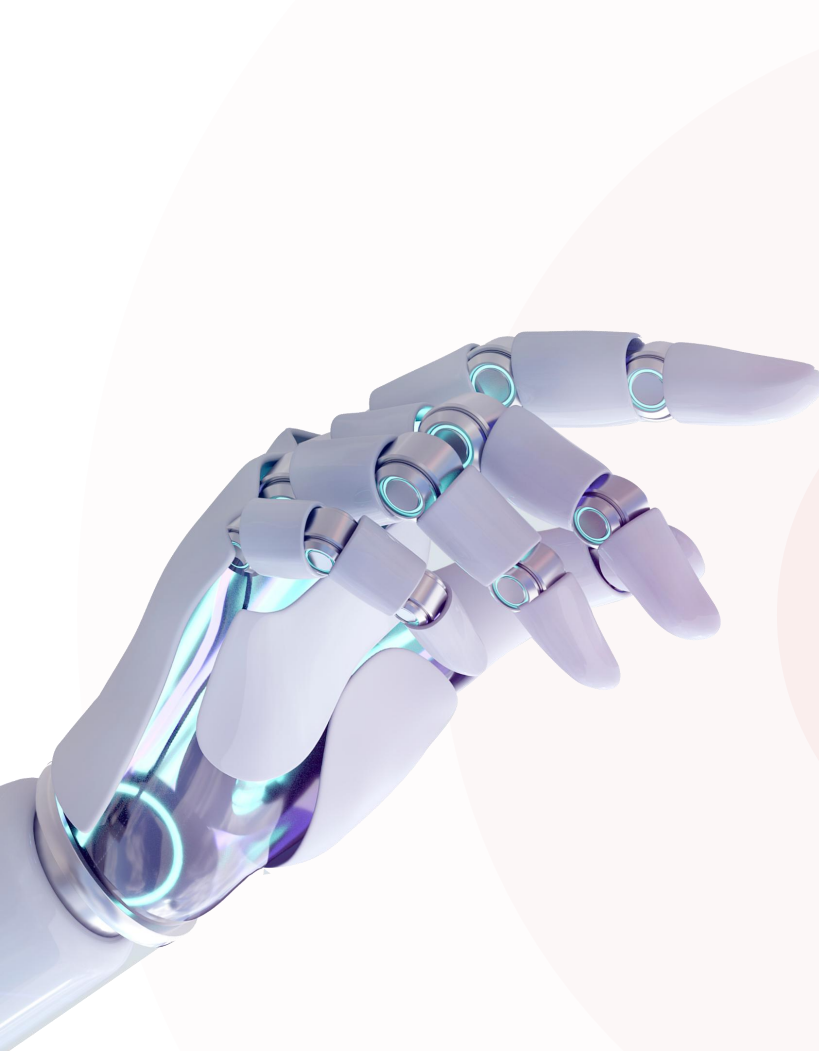
2. 关键字参数

- 调用函数时，通过 "形参名=值" 的形式传递参数，顺序没有要求。
- 如果同时存在位置参数与关键字参数，位置参数在前，关键字参数在后。

3. 两种传参方式的适用场景

- 一切以代码结构清晰明了（可读性）、便于维护（维护性）为目标。
- 如果参数比较少（不超过3个），可直接使用位置参数。
- 如果参数数量较多，建议使用关键字参数。

```
def reg_stu(name, age, gender, city):  
    print(f"注册成功! \n姓名: {name}\n年龄: {age}\n性别: {gender}\n城市: {city}")  
    return {"name": name, "age": age, "gender": gender, "city": city}  
  
stu = reg_stu("张三", 18, "男", "北京")  
print(stu)  
  
stu = reg_stu(name="李四", age=19, gender="男", city="上海")  
print(stu)  
  
stu = reg_stu("小六子", 15, gender="男", city="深圳")  
print(stu)
```



2

函数进阶

- ◆ 函数变量的作用域
- ◆ 函数参数详解
 - 传参方式
 - 默认参数
 - 不定长参数
- ◆ 匿名函数
- ◆ 案例

默认参数

- 默认参数也称为缺省参数，用于在定义函数时，为参数提供默认值，调用函数时，可以不传递有默认值的参数。

定义函数

```
def reg_stu(name, age, gender, city='北京'):
```

—————> 默认值

```
    print(f"注册成功, 姓名:{name}, 年龄:{age}, 性别:{gender}, 城市:{city}")
    return {"name": name, "age": age, "gender": gender, "city": city}
```

调用函数

```
stu = reg_stu("张三", 18, "男")
print(stu)

stu = reg_stu("赵四", 22, "男", "深圳")
print(stu)
```

注意：默认参数必须放在没有默认值的参数列表的后面，一个函数在定义时是可以设置多个默认参数的。

注意：函数调用时，如果为默认参数传递了值，则会修改默认的参数值；如果没有传递该参数，则直接使用默认值。

思考

- 需求：定义函数，根据传入的数据，计算这批数据中的最小值、最大值、平均值。

```
# 定义函数
```

```
def calc_data(参数列表):
```

```
.....
```

```
# 定义函数
```

```
def calc_data(a, b, c, d):
```

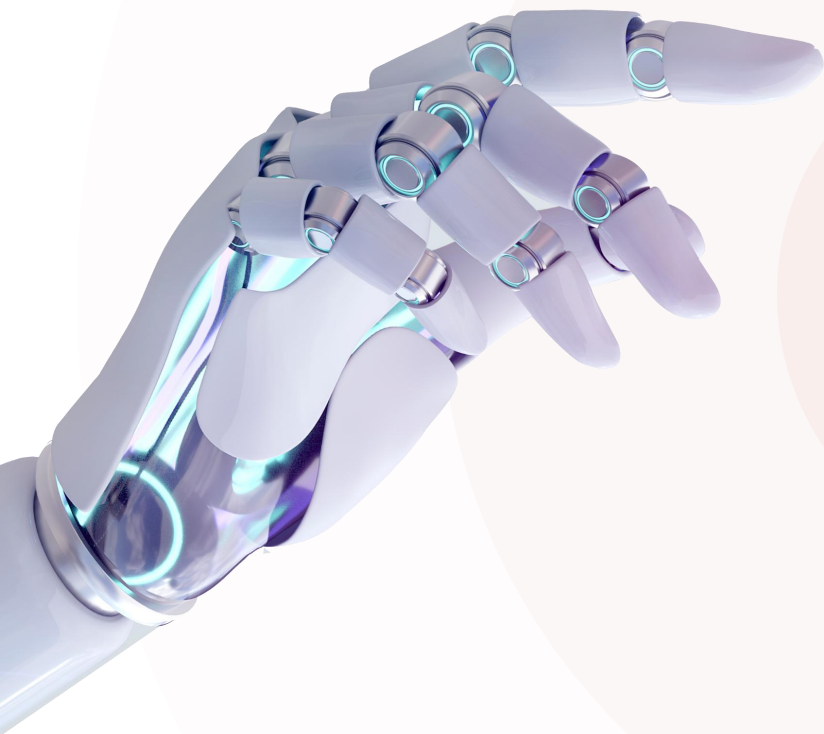
```
.....
```

问题：有多少个参数呢？ 不确定

```
min_v, max_v, avg_v = calc_data(389, 39, 29, 105)
```

```
min_v, max_v, avg_v = calc_data(15, 18, 3, 27, 89, 35)
```

```
min_v, max_v, avg_v = calc_data(100, 45, 78, 15, 183, 96, 27, 89, 35)
```



2

函数进阶

- ◆ 函数变量的作用域
- ◆ 函数参数详解
 - 传参方式
 - 默认参数
 - 不定长参数
- ◆ 匿名函数
- ◆ 案例

不定长参数

- 介绍：不定长参数也叫可变参数，用于函数定义及调用时参数个数不确定（0个或多个）的场景。
- 类型：
 - 位置传递
 - 关键字传递

不定长参数-位置传递(*args)

定义函数

```
def calc_data(*args):
```

```
    r
```

```
    r
```

```
    e
```

```
    r
```

调用函数

```
data = calc_data(10, 20, 30, 40, 50, 60, 70, 80, 90, 100)
```

```
print(data)
```

```
data = calc_data(100, 200, 300, 400, 500)
```

```
print(data)
```

注意：传递的所有匹配的位置参数都会被args变量收集，这些参数会合并封装为一个元组，args是元组类型（注意并不会封装关键字参数）。

注意：args只是约定俗成的变量名，并不是关键字，这里可以使用任何合法的变量名（如 *data）。

不定长参数-关键字传递(**kwargs)

定义函数

```
def calc_data(*args, **kwargs):
```

```
    min_data = min(args)
```

```
    max_data = max(args)
```

```
    avg_data = sum(args) / len(args)
```

```
    if kwargs.get('round'):
```

```
        avg_data = round(avg_data, kwargs.get('round'))
```

```
    return min_data, max_data, avg_data
```

```
data = calc_data(100, 200, 300, 400, round=2, count=0)
```

```
print(data)
```

```
data = calc_data(33, 11, 28, 91, 32, 75, 49)
```

```
print(data)
```

注意：参数是以 "键=值" 形式传递的关键字参数，这些 "键=值" 参数都会被kwargs接受，并合并封装为一个字典类型。

注意：kwargs只是约定俗成的变量名，并不是关键字，这里可以使用任何合法的变量名（如 **options）。

小结

1. 什么是不定长参数？

- 参数个数不确定，此时就可以使用不定长参数解决这类问题

2. 不定长参数的分类？

- `*args`：不定长位置参数，函数调用时，通过位置参数传递多个参数封装到一个元组(tuple)中
- `**kwargs`：不定长关键字参数，函数调用时，通过关键字参数传递多个参数封装到一个字典(dict)

3. `*args`与`**kwargs`的应用场景？

```
def calc_data(*args, **kwargs):
```

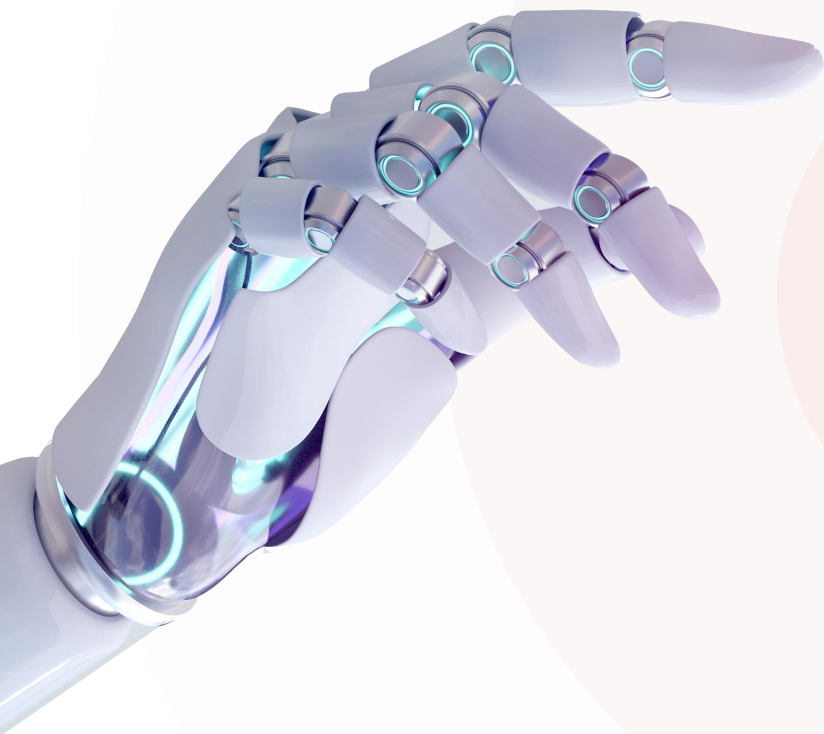
- `*args`适用于处理数量不确定的数据
- `**kwargs`适用于处理数量不确定的选项（函数的配置参数，用来定制函数的行为）

```
# 核心数据：你要什么
```

```
点奶茶("珍珠奶茶")
```

```
# 选项：你要什么样的
```

```
点奶茶("珍珠奶茶", 甜度="少糖", 冰度="去冰", 加料=["布丁", "珍珠"], 大小="大杯")
```



2

函数进阶

- ◆ 函数变量的作用域
- ◆ 函数参数详解
 - 传参方式
 - 默认参数
 - 不定长参数
 - 参数类型
- ◆ 匿名函数
- ◆ 案例

函数的参数类型

- 普通参数：数字、布尔、字符串、列表、元组、集合、字典等。
- 特殊参数：函数。

```
def circle_area(r):  
    area = 3.14 * r ** 2  
    return area
```

```
area = circle_area(10)  
print(area)
```

```
def calc_score(score_list):  
    max_s = max(score_list)  
    min_s = min(score_list)  
    avg_s = round(sum(score_list) / len(score_list), 1)  
    return max_s, min_s, avg_s
```

```
s_list = [589, 609, 605, 643, 677, 455, 477, 489, 503]  
max_score, min_score, avg_score = calc_score(s_list)
```

```
def add (x, y):  
    return x+y
```

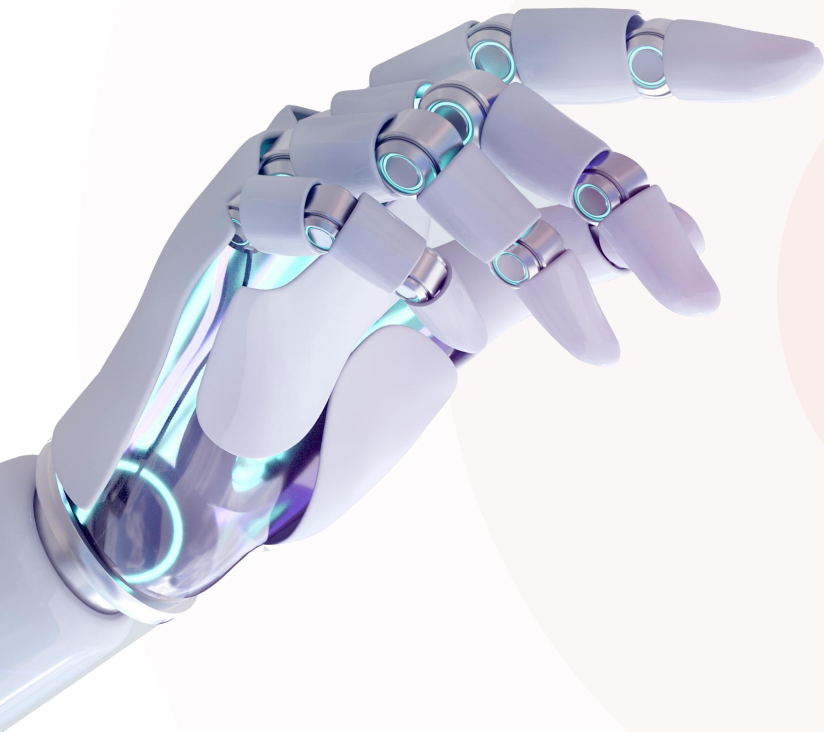
```
def subtract (x, y):  
    return x-y
```

```
def calc(x, y, oper):  
    return oper(x, y)
```

```
result = calc(10, 20, add)  
print(result)
```

传递的是实际要计算的数据

传递的是函数中封装的逻辑



2

函数进阶

- ◆ 函数变量的作用域
- ◆ 函数参数详解
- ◆ 匿名函数
- ◆ 案例

匿名函数

- 匿名函数指的是没有名称的函数，需要通过lambda表达式来声明函数，可以简化简单函数的编写（单行表达式）。

```
# 定义命名函数
```

```
def 函数名(参数列表):
```

```
    函数体...
```

```
def out_line():
```

```
    print('-----')
```

```
def add(x, y):
```

```
    return x + y
```

```
out_line()
```

```
print(add(10, 20))
```

命名函数

```
# 定义匿名函数
```

```
lambda 参数列表: 函数体
```

```
lambda : print('-----')
```

```
lambda x, y: x + y
```

匿名函数

注意：函数逻辑比较简单(单行表达式)且只在一个地方使用时，可以考虑使用匿名函数，简化书写（通常作为高阶函数的参数使用）。

注意：匿名函数中可以返回结果，也可以不返回结果。返回结果时，不需要写return，表达式的运行结果就是要返回的结果。

BRIEF 小结

1. 匿名函数的定义方式

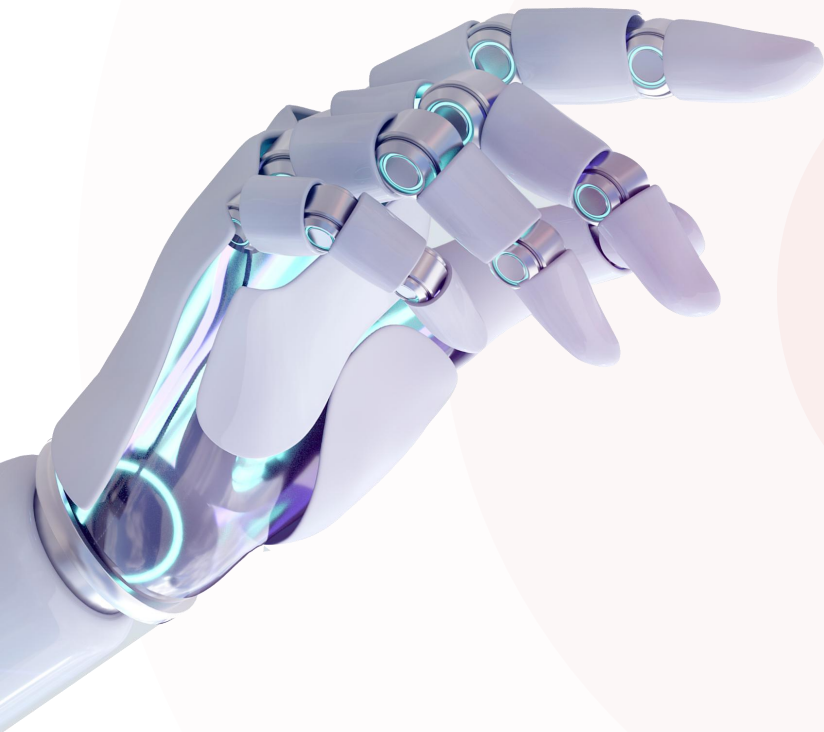
```
# 定义匿名函数
```

```
lambda 参数列表: 函数体
```

2. 命名函数与匿名函数的选择？

- 建议使用匿名函数的情况：函数逻辑简单，只在一个地方调用（常作为高阶函数的参数）
- 建议使用命名函数的情况：函数逻辑复杂，需要多步操作，需要多个地方重复使用或需要加文档说明的场景

代码的可读性和可维护性比简洁性更重要



2

函数进阶

- ◆ 函数变量的作用域
- ◆ 函数参数详解
- ◆ 匿名函数
- ◆ 案例



案例

N的阶乘



- 定义一个函数，根据传入的数字，计算该数字阶乘的结果。
- 分析：
 - 8的阶乘： $8 * 7 * 6 * 5 * 4 * 3 * 2 * 1$ $f(8) = 8 * f(7)$
 - 7的阶乘： $7 * 6 * 5 * 4 * 3 * 2 * 1$ $f(7) = 7 * f(6)$
 - 6的阶乘： $6 * 5 * 4 * 3 * 2 * 1$ $f(6) = 6 * f(5)$
 - 5的阶乘： $5 * 4 * 3 * 2 * 1$ $f(5) = 5 * f(4)$
 - 4的阶乘： $4 * 3 * 2 * 1$ $f(4) = 4 * f(3)$
 - 3的阶乘： $3 * 2 * 1$ $f(3) = 3 * f(2)$
 - 2的阶乘： $2 * 1$ $f(2) = 2 * f(1)$
 - 1的阶乘： 1 $f(1) = 1$
 - n的阶乘公式： $f(n) = n * f(n-1)$



案例

完成如下需求



1. 根据输入的班级名称，以及班级中各个学员的考试总分，统计班级的平均分，以及高于平均分的人数，低于平均分的人数。



案例

电商订单计算器



- 定义一个函数，用于根据传入的一批商品信息（商品名、价格、数量）、优惠（优惠券、积分抵扣）、运费信息计算订单的总金额。

具体规则如下：

- 优惠券需要商品金额满5000才可以使用，且优惠券金额不能超过商品总价。
- 积分抵扣需要商品总金额满5000才可以使用，100积分抵扣1元（且抵扣金额不能超过商品总价，积分只能整百抵扣）。

BRIEF 小结

● 列表推导式与生成器表达式对比

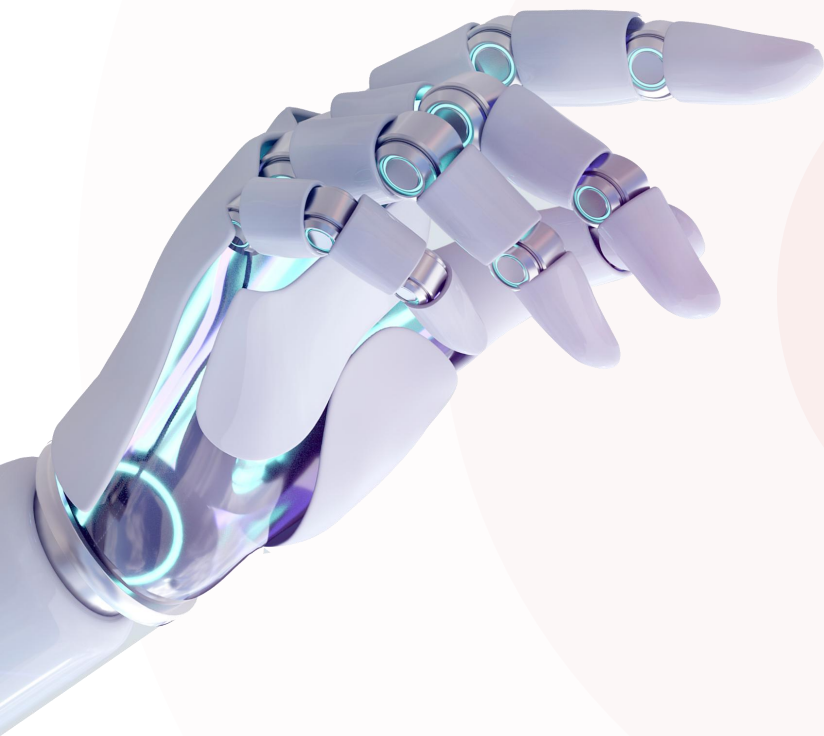
- 列表推导式：[要插入的值 for i in 数据集 if 条件]
 - 特点：立即求值，一次性生成整个列表并存入内存，占用内存空间大
 - 场景：数据量不大，需要全部立即生成的场景； 需要重复使用，多次访问的场景；
- 生成器表达式：(要插入的值 for i in 数据集 if 条件)
 - 特点：惰性求值，按需逐个生成元素，无需同时存储所有元素，节省内存
 - 场景：处理大数据集、避免一次性加载所有数据到内存；sum()/max()/min等函数的参数

小结

```
def calc_cart_total_price(*goods, coupon_price=0, point_deduction=0, express_price=0):  
  
    """  
    定义一个用于根据商品信息（商品名、价格、数量）、优惠（折扣、优惠券、积分抵扣）、运费信息计算购物车总金额的函数。  
    :param goods: 商品信息  
    :param coupon_price: 优惠金额  
    :param point_deduction: 积分抵扣金额  
    :param express_price: 运费金额  
    :return: 购物车总金额  
    """  
  
    total_price = sum(good[1] * good[2] for good in goods)  
    if total_price > 5000 and coupon_price <= total_price :  
        total_price -= coupon_price  
    if total_price > 5000 and point_deduction // 100 <= total_price:  
        total_price -= point_deduction  
    return total_price + express_price
```

参数类型不清楚

代码补全没有任何提示



3

类型注解

- ◆ 基本介绍
- ◆ 函数类型注解

类型注解

- 类型注解是Python中的一种语法特性，用于明确标识变量、函数参数和返回值的数据类型，从而使代码更清晰、更安全、更易维护。

```
# 定义变量
```

```
a = 695
```

```
score = 98.5
```

```
hobby = "Python"
```

```
flag = True
```

```
pic = None
```

```
names = ["A", "C", "E"]
```

```
phones = {"13309091111", "15209109121"}
```

```
options = {"count": 0, "total": 0}
```

```
goods = ("手机", 5999, 1)
```

```
# 定义变量
```

```
a: int = 695
```

```
score: float = 98.5
```

```
hobby: str = "Python"
```

```
flag: bool = True
```

```
pic: None = None
```

```
names: list[str] = ["A", "C", "E"]
```

```
phones: set[str] = {"13309091111", "15209109121"}
```

```
options: dict[str, int] = {"count": 0, "total": 0}
```

```
goods: tuple[str, int, int] = ("手机", 5999, 1)
```


类型推断

- 类型推断是指Python解释器自动推断出变量、表达式或函数返回值的数据类型的能力，而无需开发者显式声明。

```
# 定义变量
a = 695
score = 98.5
hobby = "Python"
flag = True
pic = None

names = ["A", "C", "E"]
phones = {"13309091111", "15209109121"}
options = {"count": 0, "total": 0}
goods = ("手机", 5999, 1)
```

```
a = 695
score = 98.5
hobby = "football"
flag = True
pic = None

names = ["张三", "李四", "王五"]
```

D:/py_code_workspace/py_project1/05. 函数进阶.py

```
names: list[str] = ["张三", "李四", "王五"]
```

goods = ("鼠标", "键盘", "USB")

注意：在对变量进行直接赋值，或者涉及到变量的运算、容器的推导等场景时，解释器会自动推导出变量的类型。

BRIEF 小结

1. 类型注解的写法？

- 变量: 数据类型 (如 `a: int`)

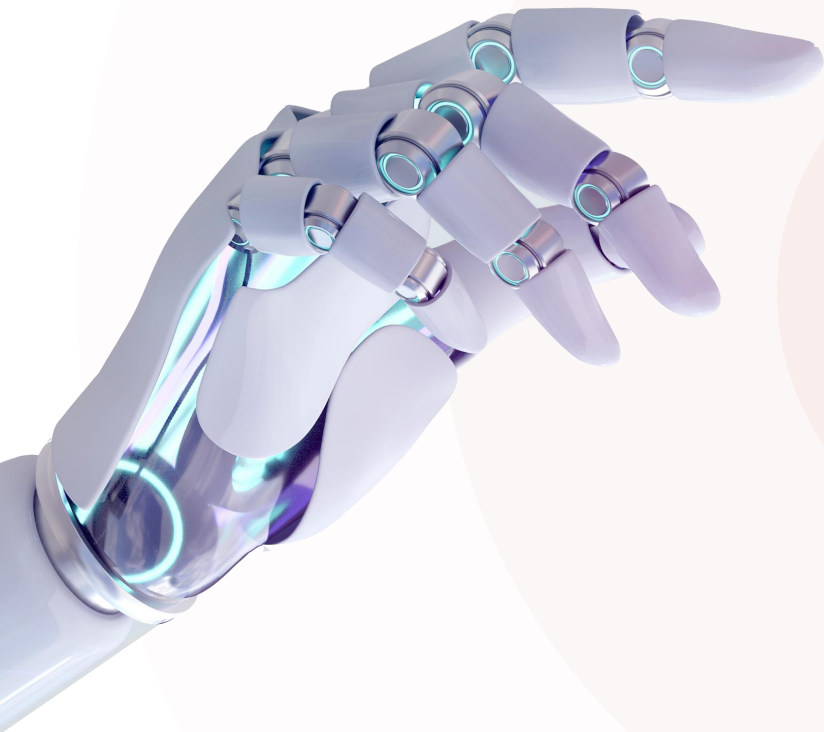
2. 常见类型的写法

- `int`、`float`、`bool`、`str`、`None`、`list`、`set`、`tuple`、`dict`
- `str | int`

3. 为什么要使用类型注解，有什么好处呢？

- 代码结构更清晰、代码逻辑更安全、易维护
- 更准确的代码自动提示
- 提前发现代码潜在问题

Python是动态类型语言,添加的类型注解只是提示,并不是强约束!!!



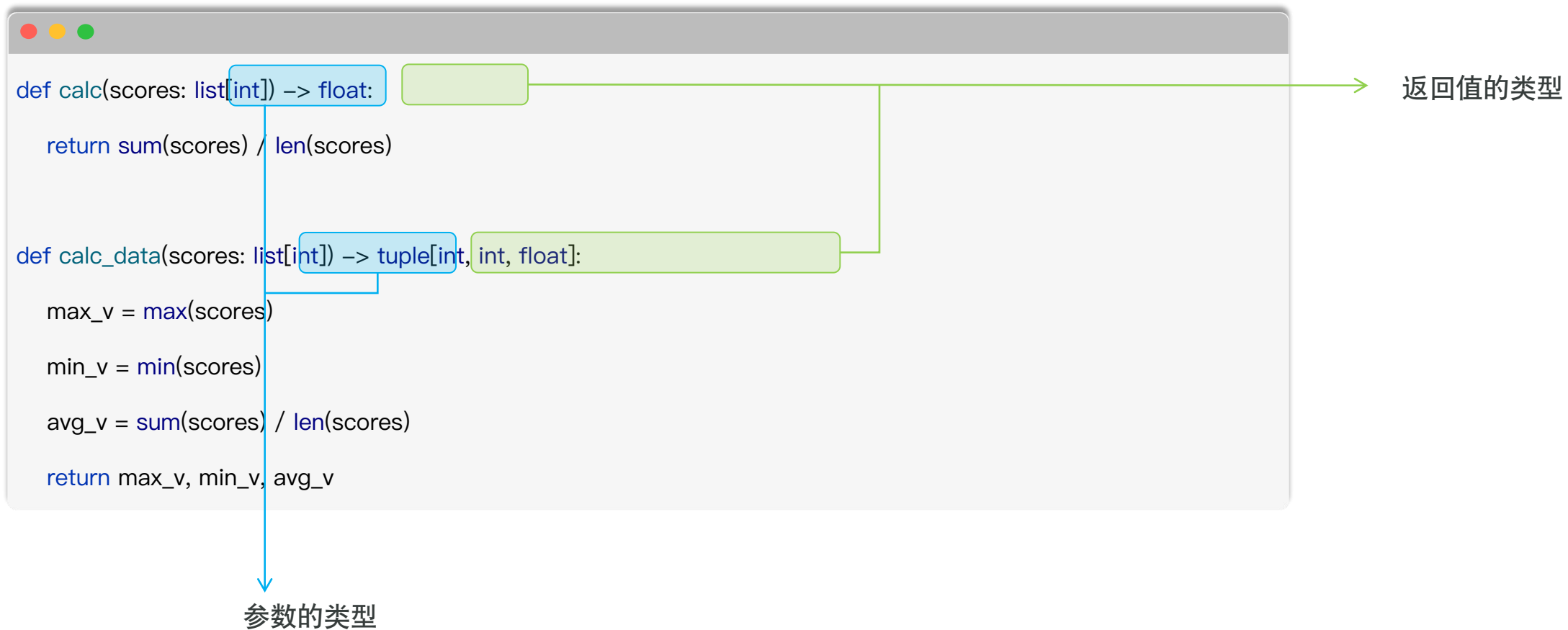
3

类型注解

- ◆ 基本介绍
- ◆ 函数类型注解

函数-类型注解

- 为函数添加类型注解，其实主要就是为函数的参数和返回值添加类型注解，具体语法如下：





案例

为函数添加合理的类型注解



- 定义一个用于根据输入的商品信息（商品名、价格、数量）、优惠（优惠券、积分抵扣）、运费信息计算购物车总金额的函数。

具体规则如下：

- 优惠券需要商品金额满5000才可以使用，且优惠券金额不能超过商品总价。
- 积分抵扣需要商品总金额满5000才可以使用，100积分抵扣1元（且抵扣金额不能超过商品总价）。

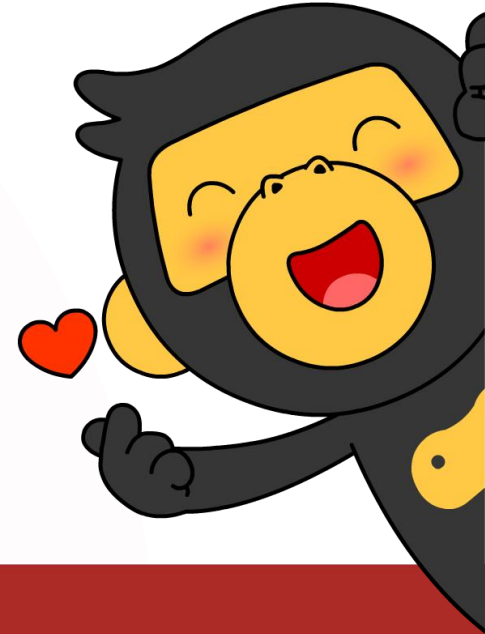
小结

1. 函数中类型注解的语法？

```
def calc_data(scores: list[int]) -> tuple[int, int, float]:  
    max_v = max(scores)  
    min_v = min(scores)  
    avg_v = sum(scores) / len(scores)  
    return max_v, min_v, avg_v
```

参数类型 返回值类型

对于需要团队协作开发和长期维护的项目，推荐使用类型注解



黑马程序员
www.itheima.com

传智教育旗下
高端IT教育品牌